Observaciones - Laboratorio 9

ISIS-1225 Estructuras de Datos y Algoritmos - Colas de prioridad

Grupo G08

Integrantes: (nombre completo, correo Uniandes y código de cada integrante)

a) ¿Qué estructura de datos subyacente se utiliza comúnmente para implementar una cola de prioridad en Python?

Un **heap binario** (montículo). Es un árbol binario casi completo que se guarda en una lista/arreglo: el padre de la posición *i* está en *i//2* y sus hijos en *2i* y *2i+1* (en nuestro lab la posición 0 queda vacía y el heap empieza en la 1). En un min-heap el padre siempre tiene prioridad menor o igual que sus hijos, así que el elemento más prioritario siempre queda en la raíz.

b) ¿Cuál es la diferencia entre una cola FIFO tradicional y una cola de prioridad?

En una cola FIFO sale primero el que llegó primero, sin importar qué tan importante sea. En una cola de prioridad sale primero el elemento con mayor prioridad (en un MinPQ, el de menor valor), sin importar cuándo llegó. El orden de llegada solo cuenta como desempate, y ni siquiera siempre.

c) ¿Qué módulo proporciona Python para trabajar fácilmente con colas de prioridad?

El módulo **heapq**, que trae funciones como *heappush*, *heappop* y *heapify* sobre una lista normal (es un min-heap). También existe **queue.PriorityQueue**, que usa heapq por dentro y además es segura para hilos (thread-safe).

d) ¿Qué ventajas tiene el uso de una cola de prioridad sobre una lista ordenada manualmente?

Con una lista ordenada, cada inserción cuesta O(n), porque hay que encontrar el lugar y correr los elementos. Con un heap, insertar cuesta O(log n). Además, el heap no necesita mantener todo ordenado, solo garantiza que el mínimo esté en la raíz, y eso es lo único que necesitamos. Menos trabajo por operación y más eficiente cuando se mezclan muchas inserciones con muchas atenciones.

e) Si dos elementos tienen la misma prioridad, ¿cómo decide la cola cuál atender primero?

Depende de la implementación. Un heap no es estable, o sea que no garantiza respetar el orden de llegada entre empates. En nuestro lab, la función de comparación usa <= (en el MinPQ), por lo que si el padre y el hijo tienen la misma prioridad no se intercambian. Eso hace que el que entró antes tienda a quedar arriba, pero no está garantizado. El enunciado dice que, con la misma hora de salida, las rutas pueden salir en cualquier orden relativo. Si se necesitara un orden estricto, se agrega un segundo criterio, por ejemplo un contador de llegada.

f) ¿Qué situaciones del mundo real se pueden modelar con colas de prioridad? Mencione al menos dos.

1) El **triage** de urgencias de un hospital, donde se atiende primero al más grave.
2) El **planificador de procesos** de un sistema operativo, que decide qué proceso usa la CPU.
3) **Algoritmos de grafos** como Dijkstra o Prim, que siempre sacan el nodo de menor costo.

4) La **programación de rutas de bus** de este lab, ordenadas por la hora de su primer servicio.
5) Simulaciones de eventos, y también el despacho de ambulancias o la atención de tickets de soporte.

g) En un sistema de atención médica, ¿cómo se puede usar una cola de prioridad para organizar a los pacientes?

Cada paciente entra a la cola con una prioridad según la gravedad que le asigna el triage (por ejemplo, 1 = crítico y 5 = no urgente) y se usa un MinPQ. Cada vez que hay un médico libre, se hace *remove()* y sale el paciente más grave, sin importar cuánto llevan esperando los demás. Si hay empate, se puede desempatar con la hora de llegada. Si un paciente empeora mientras espera, se actualiza su prioridad (como hace *improve_priority*) y el heap lo sube automáticamente.

h) ¿Cómo afectaría al rendimiento usar una lista simple en lugar de una estructura especializada como heap_pq para manejar prioridades?

Con una lista desordenada, insertar es O(1) pero encontrar y sacar el mínimo es O(n). Con una lista ordenada, sacar el mínimo es barato, pero insertar es O(n). En los dos casos, procesar n elementos cuesta en total $O(n^2)$. Con un heap, cada inserción y cada remoción cuestan O(log n), por lo que procesar n elementos cuesta O(n log n). Con pocos datos no se nota, pero con miles de registros, como los 13.903 de este lab, la diferencia sí se siente.

i) ¿Qué complejidad temporal tiene la inserción y eliminación/atención en una cola de prioridad basada en heap?

Inserción (insert): O(log n). El elemento se agrega al final y hace *swim*, subiendo como máximo la altura del árbol, que es log n.
Eliminación/atención (remove): O(log n). Se intercambia la raíz con el último elemento, se elimina el último y se hace *sink* desde la raíz, bajando como máximo log n niveles.
Consultar el primero (get_first_priority): O(1), porque siempre está en la raíz.
Buscar un valor (is_present_value / contains): O(n), porque toca recorrer el heap.